Your 10-week project is to systematically test how effective open-source web application firewalls (WAFs) are at blocking real attacks, using OWASP guidance and active web-app penetration tools (ZAP, Burp, sqlmap, Nikto, Wapiti). Below is a concrete breakdown of the topic, four clear team roles, and a week-by-week plan assuming roughly 16 hours per person per week.owasp+2

---

1. What the topic really means

You are expected to:

- **Pick one or more open-source WAFs (e.g., ModSecurity with OWASP Core Rule Set, Coraza, NAXSI, BunkerWeb).github+1**
- **Put them in front of vulnerable test applications (e.g., OWASP Juice Shop, DVWA).**
- **Use active attack tools (ZAP, Burp Suite Community, sqlmap, Nikto, Wapiti) to launch realistic attacks.**
- **Measure:**
    - **What attacks are blocked vs get through (true positives vs false negatives).**
    - **How often legitimate requests are blocked (false positives).**
    - **Any performance/latency impact and practical usability.fastly+1**

Your output by Week 10: a documented methodology, test environment, collected data/graphs, and a written + presented conclusion about how effective the chosen WAF(s) are.

---

2. Team roles (4 people)

Give each person a primary role but expect overlap and pairing.

1. **Role 1 – Project Manager & Client Liaison (PM)**

- **Owns schedule, risk log, weekly progress summaries, communication with instructor.**
- **Coordinates meetings, makes sure weekly evidence (screenshots, notes, commits) is produced.**

2. **Role 2 – WAF & Infrastructure Engineer (Infra)**
   - **Installs and configures:**
     - **Vulnerable apps (DVWA/Juice Shop).**
     - **Web servers (NGINX/Apache, or Docker).**
     - **Chosen open-source WAFs and OWASP Core Rule Set.[github]**
   - **Maintains environments and configuration snapshots.**
3. **Role 3 – Tools & Attack Lead (Offensive Tester)OWASP CRS (Official Repository) · GitHub**
   - **Becomes "power user" of OWASP ZAP, Burp Suite Community, sqlmap, Nikto, Wapiti.zaproxy+2**
   - **Designs and runs attack campaigns (SQL injection, XSS, file inclusion, etc.) aligned to OWASP Top 10.owasp+1**
4. **Role 4 – Measurement, Analysis & Documentation (Analyst)**
   - **Designs data collection templates (spreadsheets, logs).**
   - **Records test runs, calculates detection/false-positive rates, draws graphs, and drafts the report.**

You can rotate small sub-tasks for learning, but keep these primary responsibilities for accountability.

---

3. Weekly plan (10 weeks × ~16h per person)

Each week lists: focus, then tasks for each role. Adjust hours within the week as needed, but aim to show clear progression in a weekly log.

Week 1 – Understand WAFs and OWASP

Focus: Background reading + tool installation + initial planning (this directly matches your instructor's email).

- **Team outcome:**
    - **Short concept note: "What is a WAF, what is an open-source WAF, what are we investigating?"**
    - **All tools installed and launched at least once.**
- **PM**
    - **Write a 1–2 page project brief: scope, goals, assumptions, risks, and success criteria (e.g., measure detection rate for at least 3 OWASP Top 10 categories).owasp+1**
    - **Set up a shared GitHub repository.**
    - **Set fixed weekly meeting times; start a progress log template.**
- **Infra**
    - **Read OWASP WAF and WAFEC pages (what to test, typical criteria).nest.owasp+1**
    - **Propose 2–3 candidate open-source WAFs you can realistically deploy.**
    - **Install a basic web server stack locally or via Docker (NGINX/Apache with a sample app).**
      q
- **Offensive Tester**
    - **Install OWASP ZAP and Burp Suite Community, follow a basic "getting started" tutorial for each.jit+3**
    - **Install sqlmap, Nikto, Wapiti; confirm they run from command line.infosecinstitute+1**
- **Analyst**

- **Read OWASP Top 10 and identify which risks you will definitely test (e.g., Injection, Broken Access Control, XSS).[owasp]**
- **Draft a simple metrics plan: for each test, what will you log? (URL, payload, tool, WAF enabled/disabled, result, status code, time).**

Week 2 – Define evaluation design and choose WAFs

Focus: Decide exactly what you will test, how, and with which WAFs.

- **Team outcome:**
  - **Final list of WAF(s), test applications, and evaluation criteria (a 2–3 page "methodology" draft).**
- **PM**
  - **Create a 10-week Gantt-style schedule (even a simple table) showing main milestones.**
  - **Confirm expectations with instructor (number of WAFs, depth vs breadth).**
- **Infra**
  - **Deploy at least one vulnerable target app (e.g., DVWA or OWASP Juice Shop).owasp+1**
  - **Validate it works without any WAF: you can log in, perform actions.**
- **Offensive Tester**
  - **Run very simple manual attacks on the target app with no WAF:**
    - **ZAP/Burp: intercept a request, change a parameter to `' OR '1'='1` and basic XSS like `<script>alert(1)</script>`.[youtube][jit]**
  - **Document any vulnerabilities you can already trigger.**
- **Analyst**

- **Turn the metrics plan into a real spreadsheet/template (columns for tool, payload type, WAF name, pass/block, notes).**
- **Summarize OWASP WAFEC criteria that you can realistically use (e.g., detection accuracy, false positives, performance).[owasp]**

Week 3 – Build baseline and learn tools deeply

Focus: Baseline testing without WAF, and deeper practice on tools.

- **Team outcome:**
  - **Baseline dataset: what attacks succeed when there is no WAF.**
- **PM**
  - **Ensure everyone logs hours and activities for Week 1–3; start a running "lessons learned" document.**
  - **Check that risks (e.g., environment instability) are being managed.**
- **Infra**
  - **Harden the target app's logging (web server logs, app logs) so that later you can clearly see blocked vs allowed requests.**
  - **Script simple environment reset (so you can revert between tests).**
- **Offensive Tester**
  - **Use:**
    - **ZAP: spider the app, then run an active scan on key pages.zaproxy+1**
    - **Burp Community: intercept and replay/fuzz one or two critical requests.[portswigger][youtube]**
    - **sqlmap: try `--batch` against a parameter you suspect is injectable.[infosecinstitute]**

        - **Nikto and Wapiti: run basic scans against the app.aidaningram+1**
    - **Record which attacks succeed (e.g., actual SQL errors, reflected XSS pop-ups).**
- **Analyst**
    - **Capture and organize all baseline results into your spreadsheet.**
    - **Draft a short "Baseline Findings" note: which vulnerabilities are confirmed, using which tools.**

Week 4 – Deploy first WAF and repeat tests

Focus: Deploy your first open-source WAF in front of the app and see what changes.

- **Team outcome:**
    - **Working stack: Client → WAF → Web app, with logs.**
    - **First blocked vs allowed dataset for WAF #1.**
- **PM**
    - **Document the "as-built" architecture (diagram of where the WAF sits).**
    - **Verify everyone is still on track for 10-week outcomes.**
- **Infra**
    - **Install and configure WAF #1 (e.g., ModSecurity + OWASP Core Rule Set, or another open-source WAF).owasp+1**
    - **Turn on a reasonable default level of protection (not maximum, to reduce false positives initially).**
    - **Confirm normal browsing still works.**
- **Offensive Tester**

- **Repeat the exact same baseline attacks from Week 3:**
    - **ZAP active scan; Burp manual injections; sqlmap; Nikto; Wapiti.**
  - **Note which attacks that previously worked are now blocked (HTTP status codes, error messages, log entries).**

- **Analyst**
  - **Update your spreadsheet with WAF #1 results (before vs after side by side).**
  - **Start computing simple metrics: "x out of y SQLi attempts blocked," etc.**

Week 5 – Tune WAF #1 and measure false positives

Focus: Adjust rules to reduce false positives and understand configuration.

- **Team outcome:**
  - **Evidence of WAF tuning and its impact on both security and usability.**
- **PM**
  - **Prepare a short mid-project progress report (2–3 pages) summarizing to date: baseline, WAF #1 initial results, and tuning plan.**
  - **Update risk log (e.g., "We may not have time for a third WAF" etc.).**
- **Infra**
  - **Based on logs, tune WAF rules:**
    - **Lower or raise paranoia levels, add allow-lists where necessary.jit+1**
  - **Keep versions of config files (v1, v2…) to show progression.**
- **Offensive Tester**
  - **Design a small set of legitimate user scenarios (login, search, add to cart, etc.) and run them with WAF enabled to see what breaks (false positives).**

    - **Re-run a subset of attacks to see how tuning changes results.**

- **Analyst**
    - **Document specific examples of false positives (what normal action triggered a block).**
    - **Create first simple charts (even rough) of blocked vs allowed requests.**

Week 6 – Introduce WAF #2 (and possibly #3)

Focus: Add at least one more open-source WAF to compare.

- **Team outcome:**
    - **Second WAF installed and basic tests finished (even if not fully tuned).**
- **PM**
    - **Decide, with the team, whether you will compare 2 or 3 WAFs in total (depending on time).**
    - **Update schedule accordingly.**
- **Infra**
    - **Install WAF #2 (e.g., Coraza, NAXSI, or another realistic option).[owasp]**
    - **Reuse as much of the deployment pattern from WAF #1 as possible.**
- **Offensive Tester**
    - **Repeat the core baseline attacks on WAF #2 (no tuning yet).**
    - **Take note of obvious differences: does it block more or less than WAF #1?**
- **Analyst**
    - **Extend your spreadsheet to include WAF #2 columns.**

- **Log any severe usability issues or crashes you encounter.**

Week 7 – Advanced attacks and WAF evasion

Focus: Try more sophisticated payloads and basic WAF-bypass techniques.

- **Team outcome:**
  - **Evidence that you attempted evasions (e.g., encoded payloads, altered HTTP headers).**
- **PM**
  - **Ensure ethical boundaries are clear (testing only on lab environments).**
  - **Start drafting the "Methodology" and "Ethics and Limitations" sections of your final report.**
- **Infra**
  - **Stabilize both WAF environments so repeated advanced tests don't break them.**
  - **Improve log collection (centralized logs if possible).**
- **Offensive Tester**
  - **Use:**
    - **sqlmap tamper scripts and WAF detection options to see if it recognises and bypasses your WAFs.vaadata+1**
    - **ZAP and Burp to craft slightly obfuscated attacks (URL-encoded, extra parameters, header changes).**
  - **Document any successful bypasses in detail.**
- **Analyst**

- **Categorize each attack as: blocked, detected but allowed, or fully bypassed.**
- **Start drafting a "Results" chapter outline with key comparisons.**

Week 8 – Performance and stability testing

Focus: Measure overhead and stability of each WAF.

- **Team outcome:**
  - **Approximate performance impact metrics and qualitative stability notes.**
- **PM**
  - **Confirm what performance metrics are feasible (simple is OK: page load times, number of requests before slow-down).**
  - **Ensure all configs, scripts, and data are backed up.**
- **Infra**
  - **Use simple load-testing tools (ab, hey, or small scripts) to send normal traffic with and without each WAF.**
  - **Measure approximate response time differences.**
- **Offensive Tester**
  - **While load is running, perform light attack traffic and see if WAF behaviour changes under load.**
  - **Note any timeouts or crashes.**
- **Analyst**
  - **Integrate performance numbers into your dataset and charts (e.g., average response time with vs without WAF).**
  - **Draft the performance section of the report.**

Week 9 – Consolidate data and write report

Focus: Turn all results into a structured technical report.

- **Team outcome:**
    - **Full draft of the final report (even if rough), with figures and tables.**
- **PM**
    - **Own the report structure: Introduction, Background, Methodology, Results, Discussion, Conclusion, Future Work.**
    - **Ensure all sections have owners and due dates.**
- **Infra**
    - **Write the "Environment and Configuration" appendix: versions, diagrams, key configuration snippets (no secrets).**
    - **Confirm all experiments are reproducible from your notes.**
- **Offensive Tester**
    - **Write the "Testing Methodology" subsection: how you used ZAP, Burp, sqlmap, Nikto, Wapiti, and why active scanning was chosen.aidaningram+3**
    - **Provide example payloads and attack flows.**
- **Analyst**
    - **Finalize all tables/graphs (detection rates, false positives, bypass examples, performance).**
    - **Write the main "Results" and "Analysis" narrative.**

Week 10 – Polish, rehearse, and deliver

Focus: Finalise everything and prepare for presentation/submission.

- **Team outcome:**
    - **Polished report, slide deck, and rehearsed presentation/demonstration.**
- **PM**
    - **Check marking rubric and align report + slides accordingly.**
    - **Run a full team rehearsal, time the presentation, gather feedback.**
- **Infra**
    - **Prepare a stable demo environment (if you are going to demonstrate a blocked vs unblocked attack live).**
    - **Take screenshots as backup in case live demo fails.**
- **Offensive Tester**
    - **Script 1–2 short demo scenarios:**
        - **Attack without WAF (success).**
        - **Same attack with WAF, showing block/log entry.**
    - **Prepare to explain the tools briefly (ZAP/Burp/sqlmap/Nikto/Wapiti).[youtube]zaproxy+2**
- **Analyst**
    - **Polish visuals in the report and slides (clear legends, readable text).**
    - **Write the abstract and final conclusion, highlighting which WAF(s) were most effective and under what conditions.**

If you like, next I can turn this into a one-page "project charter" or a table you can hand in or use directly as your weekly log.